Планировщик отпусков

КОРПОРАТИВНЫЙ ГРАФИК ОТПУСКОВ И СОГЛАСОВАНИЙ

Описание архитектуры

Платформа, компоненты, решения и план развития · версия 0.6.0

29 октября 2026

© Alexander Yurasov

Содержание

1 О документе 4

2 Общие сведения 5

2.1 Назначение и класс системы 5

2.2 Архитектурные принципы 5

2.3 Показатели кодовой базы 6

3 Контекст системы 7

4 Контейнеры и развёртывание 8

4.1 Состав контейнеров 8

4.2 Поставка и окружения 10

4.3 Масштабирование 10

5 Серверная часть 12

5.1 Слои и компоненты 12

5.2 Ключевые сервисы 14

5.3 Обработка запроса 16

6 Клиентская часть 18

7 Модель данных 20

7.1 Организация 20

7.2 Отпуска и режим работы 21

7.3 Служебные данные 21

7.4 Кадровый учёт 22

7.5 Жизненный цикл заявки 22

8 Ключевые сценарии 23

8.1 Вход и продление сессии 23

8.2 Подача заявки 24

8.3 Согласование и отклонение 25

8.4 Построение графика 26

9 Безопасность 27

9.1 Аутентификация и сессии 27

9.2 Защита запросов 29

9.3 Модель авторизации 29

9.4 Журналирование и аудит 30

9.5 Чек-лист безопасности 30

10 Программный интерфейс 33

11 Наблюдаемость и эксплуатация 34

12 Технологический стек 35

13 Ключевые решения 36

14 План развития 38

1 О документе

Документ описывает архитектуру «Планировщика отпусков» версии 0.5.0: назначение и границы системы, состав компонентов и их взаимодействие, модель данных, программный интерфейс, механизмы безопасности, развёртывание, эксплуатацию и план развития. Документ предназначен для архитекторов, разработчиков, инженеров сопровождения и службы информационной безопасности.

СВЯЗАННЫЙ ДОКУМЕНТ	СОДЕРЖАНИЕ
0 — Ревизия функциональности	Что было в прототипе и что стало целевым составом
2 — Спецификация API v1	Протокол, каталог операций и ошибок, матрица прав
3 — План развития и бэклог	Версии, эпики и истории с критериями приёмки
4 — Модель данных	Сущности, словарь данных, инварианты, миграции
7 — Стенды, CI/CD и выпуск	Окружения, конвейеры, порядок выпуска
11 — SBOM и лицензии	Состав сторонних компонентов для реестра отечественного ПО
Руководство по развёртыванию	Установка, обновление, резервное копирование

Источники истины, на которые опирается документ: контракт `contracts/openapi-vacation-api-v1.yaml`, схема `server/prisma/schema.prisma` и исходный код ветки `v0.5.0`. Решения, принятые с обоснованием и альтернативами, — `docs/adr/`. Показатели кодовой базы ниже сформированы автоматически (`tools/docs/gen_tables.py`), а соответствие контракта и реализации проверяется тестом в CI.

2 Общие сведения

2.1 Назначение и класс системы

«Планировщик отпусков» — корпоративное веб-приложение для планирования и согласования ежегодных отпусков. Оно ведёт график отпусков всей организации, заявки сотрудников с согласованием руководителем, балансы дней по нормам Трудового кодекса, режим работы офис/удалёнка, аналитику пересечений внутри отделов и неизменяемый журнал всех изменений.

По архитектурному типу это **клиент-серверное веб-приложение**: одностраничный клиент (SPA) и REST API, устанавливаемые на серверы организации (on-premise) в контейнерах Docker. Целевой масштаб — от 1000 до 5000 пользователей.



Рис. 1. Роли и их возможности

2.2 Архитектурные принципы

1. **Сервер решает, клиент показывает.** Правила (конечный автомат заявки, балансы, лимиты, права, конфликты) реализованы только в серверных сервисах. Проверки в клиенте — подсказки для удобства, а не защита.
2. **Контракт прежде кода.** REST API v1 описан в `contracts/` и меняется через ревью владельца; тест сверки в CI не даёт реализации и контракту разойтись.
3. **Одна операция — одна транзакция.** Изменение данных, запись в историю заявки, журнал аудита и уведомления фиксируются в одной транзакции PostgreSQL.
4. **Чистые вычисления отдельно от ввода-вывода.** Баланс, интервалы и аналитика пересечений — модули без БД и без «текущего времени»: их проверяют модульные тесты, а дата расчёта передаётся аргументом.

5. **Никаких временных решений.** Зрелый стек с первой версии: PostgreSQL, TypeScript strict, контейнеры, миграции, CI с e2e-тестами.
6. **Зависимости — под контролем.** Каждая новая библиотека обосновывается в `docs/decisions.md` и проходит лицензионный гейт SBOM: copyleft и неизвестные лицензии блокируют слияние.
7. **Секреты — только в окружении.** В репозитории нет паролей и ключей; сервер не стартует со слабым `JWT_SECRET`.

2.3 Показатели кодовой базы

ПОКАЗАТЕЛЬ	ЗНАЧЕНИЕ
Сервер: файлов TypeScript / строк	82 / 7 954
Сервер: контроллеров / операций API	20 / 83
Модель данных: таблиц / миграций	24 / 12
Клиент: файлов TS и Vue / строк	50 / 14 437
Клиент: экранов	13
Тесты сервера: файлов / сценариев	23 / 194
Тесты клиента (модульные и e2e): файлов / сценариев	9 / 44

3 Контекст системы

Пользователи работают с системой через браузер по HTTPS. Внешние системы организации подключаются через стандартные протоколы: SMTP, OpenID Connect, файлы выгрузки.

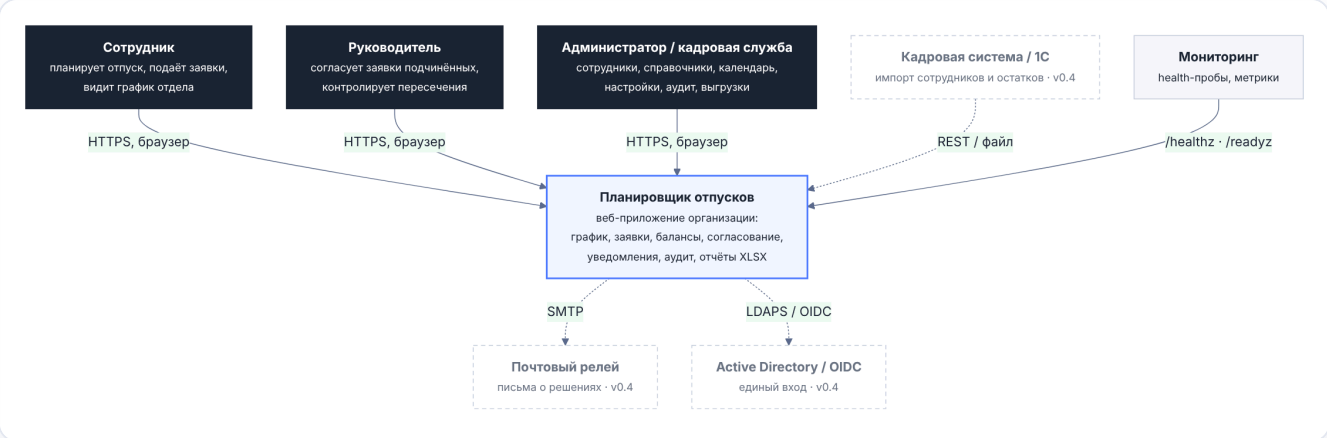


Рис. 2. Диаграмма контекста (уровень 1 C4)

УЧАСТНИК	ВЗАИМОДЕЙСТВИЕ
Сотрудник	Планирует отпуск, подаёт и переносит заявки, смотрит график отдела и свой график работы
Руководитель	Согласует заявки подчинённых, видит пересечения и аналитику отдела, ведёт режим работы команды
Администратор и кадровая служба	Сотрудники, справочники, производственный календарь, правила организации, журнал аудита, выгрузки
Мониторинг	Опрашивает `/healthz` (процесс жив) и `/readyz` (доступна БД); Prometheus собирает `/metrics`
Почтовый релей	Письма о заявках и еженедельные сводки (SMTP)
Провайдер единого входа	OpenID Connect: Keycloak, ADFS, Blitz Identity и др.
Кадровая система	Выгрузки сотрудников и остатков для импорта (XLSX, CSV, JSON)

4 Контейнеры и развёртывание

4.1 Состав контейнеров

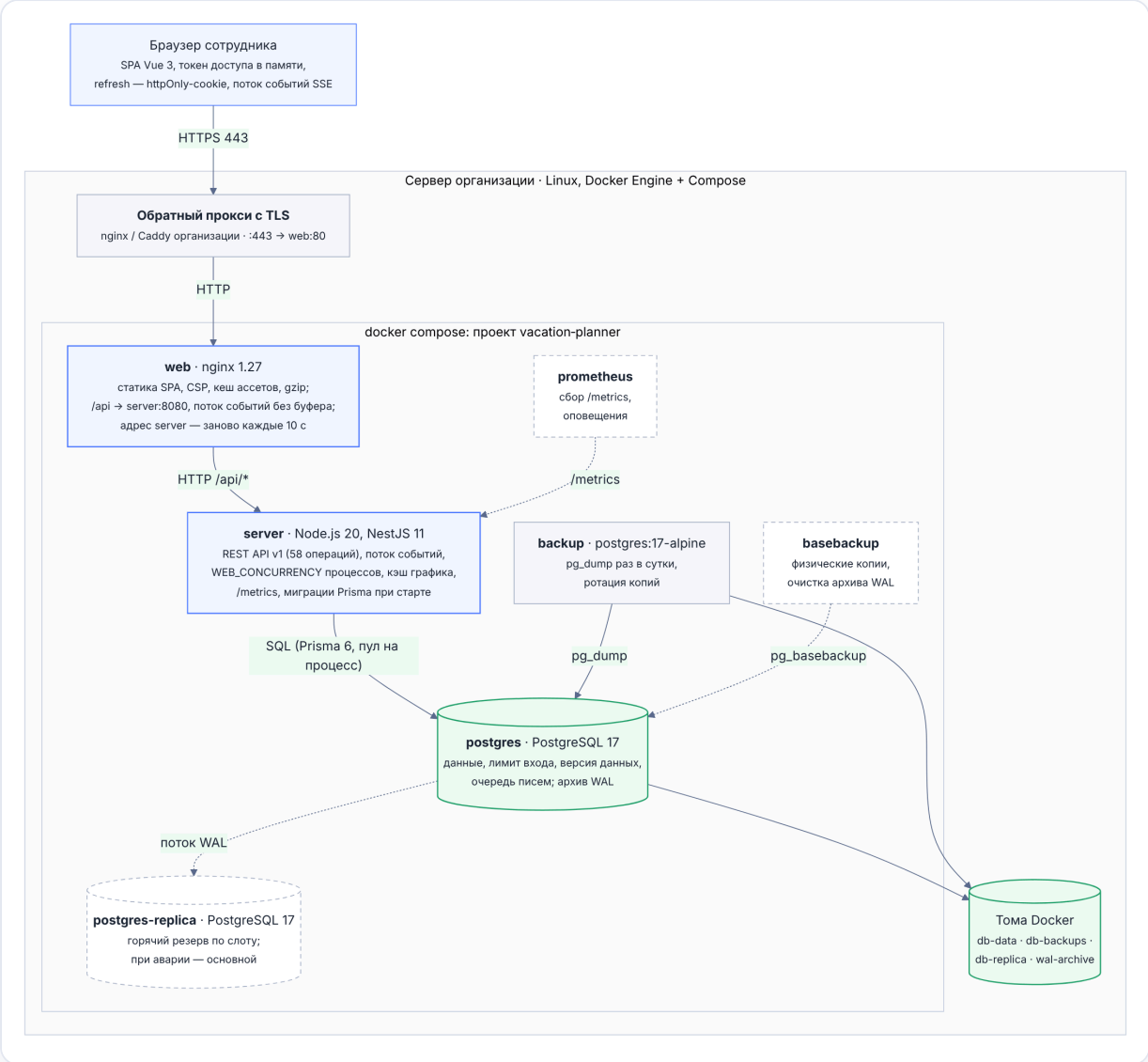


Рис. 3. Диаграмма контейнеров (уровень 2 C4)

КОНТЕЙНЕР	ОБРАЗ	НАЗНАЧЕНИЕ
`web`	nginx 1.27 + статическая сборка Vite	Отдаёт SPA с CSP и заголовками безопасности, кеширует хешированные ассеты, проксирует `/api` на `server:8080` с `X-Forwarded-For`; поток событий — без буферизации; адрес `server` разрешается заново каждые 10 с

КОНТЕЙНЕР	ОБРАЗ	НАЗНАЧЕНИЕ
`server`	Node.js 20, NestJS 11	REST API v1, бизнес-логика, поток событий, метрики; `WEB_CONCURRENCY` процессов; миграции Prisma при старте контейнера
`postgres`	PostgreSQL 17	Все данные; том `db-data`
`backup`	postgres:17-alpine	`pg_dump` по расписанию (по умолчанию раз в сутки), ротация копий 14 дней; том `db-backups`
`postgres-replica`	PostgreSQL 17	Горячий резерв: потоковая репликация по слоту, чтение; при аварии повышается до основного (`docker-compose.replica.yml`)
`basebackup`	postgres:17-alpine	Ежедневные физические копии и очистка архива WAL — восстановление на момент времени (`docker-compose.replica.yml`)
`prometheus`	Prometheus 3	Сбор `/metrics` и правила оповещений (профиль `monitoring`); панель — в Grafana организации

Перед стендом ставится обратный прокси организации с TLS (nginx или Caddy). Состояние сессий в сервере не хранится: access-токен — JWT, refresh-токены — в таблице PostgreSQL. Всё общее для процессов и экземпляров — лимит входа, версия данных графика, очередь писем, отметки фоновых задач — тоже в PostgreSQL (ADR-012), поэтому сервер масштабируется процессами и экземплярами без «липких» сессий и без отдельного хранилища.

4.2 Поставка и окружения

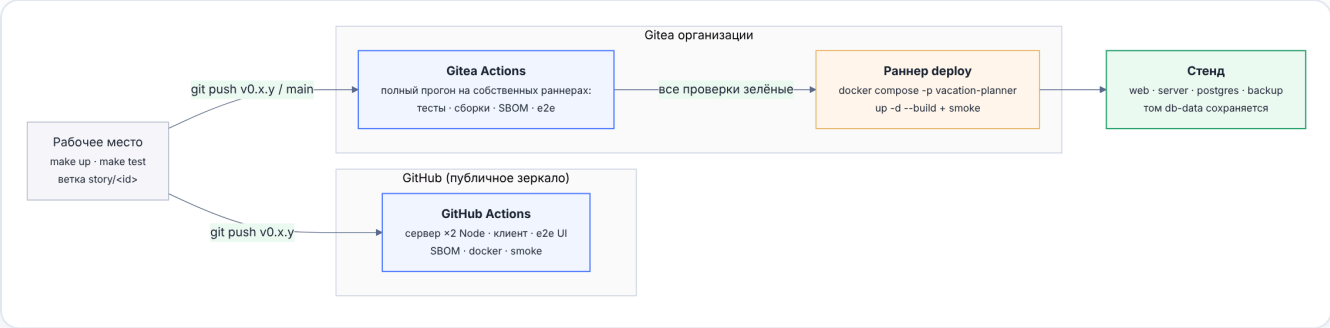


Рис. 4. Поставка: от рабочего места до стенда

Код ведётся в двух репозиториях-зеркалах: GitHub (публичный) и Gitea организации. Оба запускают одинаковый набор проверок; в Gitea после зелёных проверок раннер `deploy` разворачивает стенд (`docker compose -p vacation-planner up -d --build`) и выполняет дымовые пробы. Подробно — документ 7.

4.3 Масштабирование

НАГРУЗКА	РЕШЕНИЕ
До 1000 пользователей	Один сервер: 2 vCPU, 4 ГБ ОЗУ, PostgreSQL в том же compose, `WEB_CONCURRENCY=auto`
До 5000 пользователей	4 vCPU и 8 ГБ ОЗУ под `server`, PostgreSQL — на отдельном узле или с репликой; при необходимости второй экземпляр `server` за балансировщиком. Проверено нагрузкой: 500 одновременных пользователей при 5000 в базе (`docs/progress/load-v0.5.0.md`; v0.6.0 — `load-v0.6.0.md`)
Пики начала дня (×5)	Ответ графика кэшируется по версии данных (ETag, `304 Not Modified`); тяжёлые чтения ограничены периодом и фильтром по отделу; выгрузка XLSX пишется потоком и уступает процессор другим запросам

МЕХАНИЗМ	ЗАЧЕМ
Кластер процессов (`WEB_CONCURRENCY`)	Node.js обслуживает запросы одним ядром; главный процесс запускает по процессу на ядро и перезапускает упавшие

МЕХАНИЗМ	ЗАЧЕМ
Асинхронный scrypt	Хеширование пароля — в пуле потоков libuv: 100 мс расчёта не останавливают остальные запросы процесса
Пул соединений на процесс	min(10, 40 / процессов), не меньше 3: процессы одного экземпляра держат не больше ~40 соединений с БД
Версия данных графика	Отложенные триггеры таблиц графика увеличивают счётчик при фиксации, один раз за транзакцию (16 строк — без конкуренции за одну); ответ `/schedule` кэшируется по версии и отдаётся `304` по ETag
Лимит входа в PostgreSQL	Окна по минутам в нежурналируемой таблице: одинаковый лимит при любом числе процессов и экземпляров
Поток событий вместо опроса	Уведомления и «данные изменились» приходят по SSE; процесс опрашивает БД одним запросом раз в 2 с на всех своих подключённых (ADR-013)
Виртуальный список графика	Браузер рисует только видимые строки: 5000 сотрудников — 1,8 тыс. узлов DOM вместо 65 тыс.

5 Серверная часть

5.1 Слои и компоненты

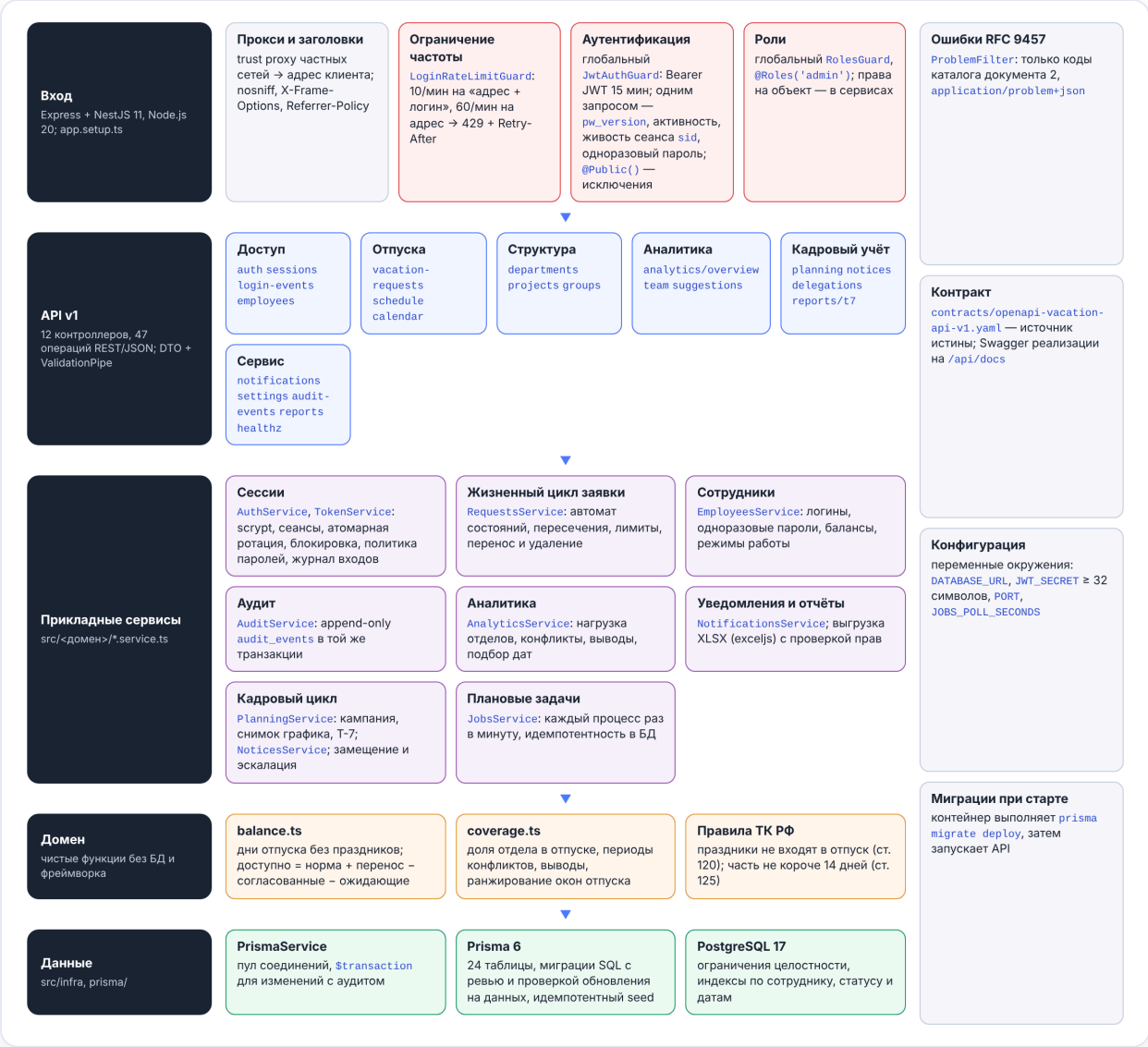


Рис. 5. Слои сервера (уровень 3 C4)

Сервер — модульный монолит NestJS: каждый домен в каталоге `server/src/<домен>/` содержит тонкий контроллер (DTO-валидация, вызов сервиса) и сервис с логикой.

МОДУЛЬ	ОТВЕТСТВЕННОСТЬ
`auth`	Вход по паролю и через единый вход OIDC (`oidc.ts`), выход, обновление и смена пароля, личные настройки; `TokenService` (JWT HS256), глобальные guards, ограничение частоты входа; `access.ts` — служебные роли (администратор, кадровая служба); сеансы, блокировка после неудачных попыток, журнал входов (`login-journal.service.ts`), политика паролей (`password-policy.ts`), одноразовый пароль (ADR-015)
`app.setup.ts`	Общая настройка HTTP: trust proxy, cookie, заголовки, валидация — одна для продакшена и тестов
`employees`	Сотрудники, логины и одноразовые пароли, балансы, режим работы (шаблон и исключения)
`requests`	Заявки: конечный автомат, пересечения, лимиты, перенос и удаление; `balance.ts` — чистые функции баланса; `approval-scope.ts` — кто решает по заявке (руководитель, руководитель отдела, заместитель, вышестоящий по эскалации); `escalation.service.ts` — передача заявки без решения вышестоящему
`delegations`	Замещение руководителя на период: назначение руководителем или кадровой службой, отмена
`notices`	Извещение о начале отпуска за две недели (ст. 123 ТК РФ): плановая задача, ознакомление, отзыв при переносе и отмене, журнал и выгрузка для кадровой службы
`planning`	Кампания планирования на год, прогресс по отделам, напоминания, утверждение графика со снимком (ADR-016), форма Т-7 (`t7.ts`)
`jobs`	Плановые задачи в каждом процессе; идемпотентность — в PostgreSQL (ADR-017)
`schedule`	График отпусков за период с конфликтами отделов: сборка простым SQL, кэш по версии данных, ETag
`events`	Поток событий (SSE): билеты, подключения, опрос новых уведомлений и версии данных

МОДУЛЬ	ОТВЕТСТВЕННОСТЬ
`analytics`	Нагрузка отделов, периоды конфликтов, выводы руководителю, подбор дат; `coverage.ts` — чистые функции
`directories`	Отделы, проекты, личные группы
`calendar`	Производственный календарь: праздники ст. 112 и переносы выходных; правка кадровой службой
`imports`	Импорт сотрудников и остатков из XLSX/CSV и JSON; `plan.ts` — чистая функция плана
`mail`	Письма о заявках (outbox в транзакции события), фоновая отправка с повторами, еженедельная сводка руководителям
`settings`	Правила организации: порог конфликта, политика лимита, норма дней, безопасность входа, срок эскалации, срок извещения, реквизиты и подписанты формы Т-7
`notifications`	Уведомления в интерфейсе и постановка писем в очередь
`audit`	Неизменяемый журнал; запись только внутри транзакции изменения
`reports`	Выгрузка отпусков в XLSX (журнал извещений и форма Т-7 — в своих модулях)
`health`	Пробы `/healthz` и `/readyz`, метрики `/metrics`
`infra`	`PrismaService`, `ProblemException` и фильтр RFC 9457; `metrics.ts` — реестр метрик и сложение процессов кластера; `concurrency.ts` — число процессов и пул; `data-version.ts` — версия данных и кэш ответов

5.2 Ключевые сервисы

RequestsService реализует жизненный цикл заявки. Создание проверяет права (за себя, за подчинённого, администратор), даты (формат, порядок, не в прошлом), пересечения с собственными заявками и лимит дней. Заявка, оформленная руководителем за подчинённого, сразу согласована. Перенос согласованного отпуска переводит заявку на согласование с

сохранением прежних дат в `change_from_*`; отклонение переноса возвращает прежние даты. Запрос удаления помечает заявку `pending_deletion`; согласование удаляет её, а событие остаётся в истории и аудите.

Баланс считается по правилам ТК РФ: праздничные нерабочие дни внутри отпуска не входят в число дней отпуска (ст. 120), одна из частей должна быть не короче 14 календарных дней (ст. 125). Доступно для планирования = норма + перенос − согласованные дни − дни на согласовании. При жёсткой политике лимита сервер отклоняет заявку сверх доступного (`409 limit-exceeded`).

AnalyticsService строит аналитику отдела на чистых функциях `coverage.ts`:

- доля отдела в отпуске по месяцам (только рабочие дни; отдельно — с учётом заявок на согласовании);
- периоды конфликтов, когда в отпуске не меньше порога отдела (выходные не разрывают период);
- выводы руководителю: ближайшие конфликты, пиковый и самый спокойный месяц, нераспределённые дни, сотрудники без части в 14 дней, риски малочисленного отдела;
- **подбор дат**: для заданной длины отпуска перебираются окна периода поиска; окно продлевается на праздники, окна с превышением порога и пересечением с собственными отпусками исключаются, остальные ранжируются по нагрузке на отдел, бонусу за праздники внутри и старту в понедельник; выдаётся до пяти непересекающихся вариантов с объяснением.

ImportsService строит план импорта той же чистой функцией для предпросмотра и применения: строка сопоставляется с сотрудником по логину, без логина — по ФИО; новые сотрудники получают логин транслитом, новые отделы создаются, руководитель может быть создан этим же файлом. Пустая ячейка значение не меняет, поэтому повторная загрузка того же файла ничего не меняет. Применение — одна транзакция с аудитом каждой записи и только при отсутствии ошибок в плане.

MailService реализует шаблон outbox: письмо пишется в таблицу `email_outbox` в той же транзакции, что и уведомление, и отправляется фоном. Письма разбираются с `FOR UPDATE SKIP LOCKED` и арендой на 5 минут, поэтому несколько экземпляров сервера не отправят письмо дважды, а упавший процесс не удерживает его. Неудачная отправка повторяется через 1, 5, 30 минут, 2 и 12 часов. Еженедельная сводка запускается один раз за неделю — по строке `job_runs`.

ScheduleService собирает график четырьмя запросами SQL (сотрудники, отпуска, шаблоны и исключения режима работы) и считает конфликты отделов счётчиками по дням. Контроллер сначала читает версию данных, затем данные: так версия не может опередить данные. Готовый ответ кэшируется в памяти процесса (до 64 МБ) по ключу «версия + параметры»; запрос с `If-None-Match` текущей версии получает `304` без тела.

Согласование считается в одном месте — `approval-scope.ts`: очередь «ждут моего решения» и право решения строятся из одних правил. Решает руководитель сотрудника, а у сотрудника без руководителя — руководитель отдела; руководитель отдела вправе решать по любому

сотруднику отдела; заместитель на период замещения — всё то же за руководителя; вышестоящий — по заявке, переданной ему эскалацией. По своей заявке не решает никто, кроме администратора. **EscalationService** раз в минуту находит заявки без решения дольше `escalationDays` (правило организации, 0 — выключено) и передаёт их руководителю согласующего; если выше никого нет — уведомляет кадровую службу. Повторно заявка не передаётся; перенос и запрос удаления начинают отсчёт заново.

NoticesService раз в минуту извещает сотрудников о согласованных ежегодных отпусках, начинающихся в ближайшие `noticeDays` дней (не меньше 14): уведомление и письмо со ссылкой на «Мой отпуск», где сотрудник подтверждает ознакомление. Отпуск, согласованный позже чем за две недели, получает извещение сразу с пометкой о праве на перенос (ст. 124). Уникальный ключ «заявка + дата начала» не даёт известить дважды; перенос на новые даты — новое извещение, а прежнее отзывается; отмена отпуска и увольнение тоже отзывают извещение.

PlanningService ведёт кампанию планирования на год: открытие со сроком рассылает приглашение всем штатным сотрудникам одной пачкой, прогресс считается по отпускам, начинающимся в году (согласованным и на согласовании): запланировано ли что-то, выполнена ли норма, есть ли часть не короче 14 дней. Напоминание уходит тем, кто не спланировал полностью: вручную — не чаще раза в час, автоматически — один раз за три дня до срока. Утверждение графика фиксирует снимок согласованных отпусков с реквизитами документа (ADR-016); форма Т-7 строится из снимка, а переносы и отмены после утверждения — по текущим заявкам.

JobsService в каждом процессе раз в `JOBS_POLL_SECONDS` (по умолчанию 60) проходит плановые задачи: извещения, эскалацию, напоминания о планировании, очистку журнала входов. Повтор безопасен за счёт ограничений в БД, а не выбора главного процесса (ADR-017).

EventsService держит открытые потоки SSE. Раз в 2 секунды процесс одним запросом находит новые уведомления своих подключённых сотрудников и проверяет версию данных; сотрудник получает `notifications` с числом непрочитанных, все — `data` при изменении графика (не чаще раза в 15 с). Каждые 25 секунд — пустой комментарий, чтобы прокси не закрыл простаивающее соединение.

5.3 Обработка запроса



Рис. 6. Маршрут запроса через защиту и валидацию

Каждый запрос `/api/v1/*` проходит ограничение частоты (для входа), глобальную аутентификацию (кроме маршрутов `@Public()`), сверку версии пароля в токене, активности сотрудника и живости сеанса (смена пароля, увольнение или завершение сеанса отзывает выданные токены; процесс помнит результат проверки 10 секунд и сбрасывает его сразу, если изменение прошло через него), проверку одноразового пароля (до смены доступны только смена пароля и выход — `403 auth/password-change-required`), проверку роли и DTO-валидацию с `whitelist`. Все эти сведения guard получает одним запросом к БД. Права на конкретный объект (своя заявка, подчинённый) проверяет сервис. Ошибка любого уровня возвращается как `application/problem+json` с кодом из каталога (документ 2).

6 Клиентская часть

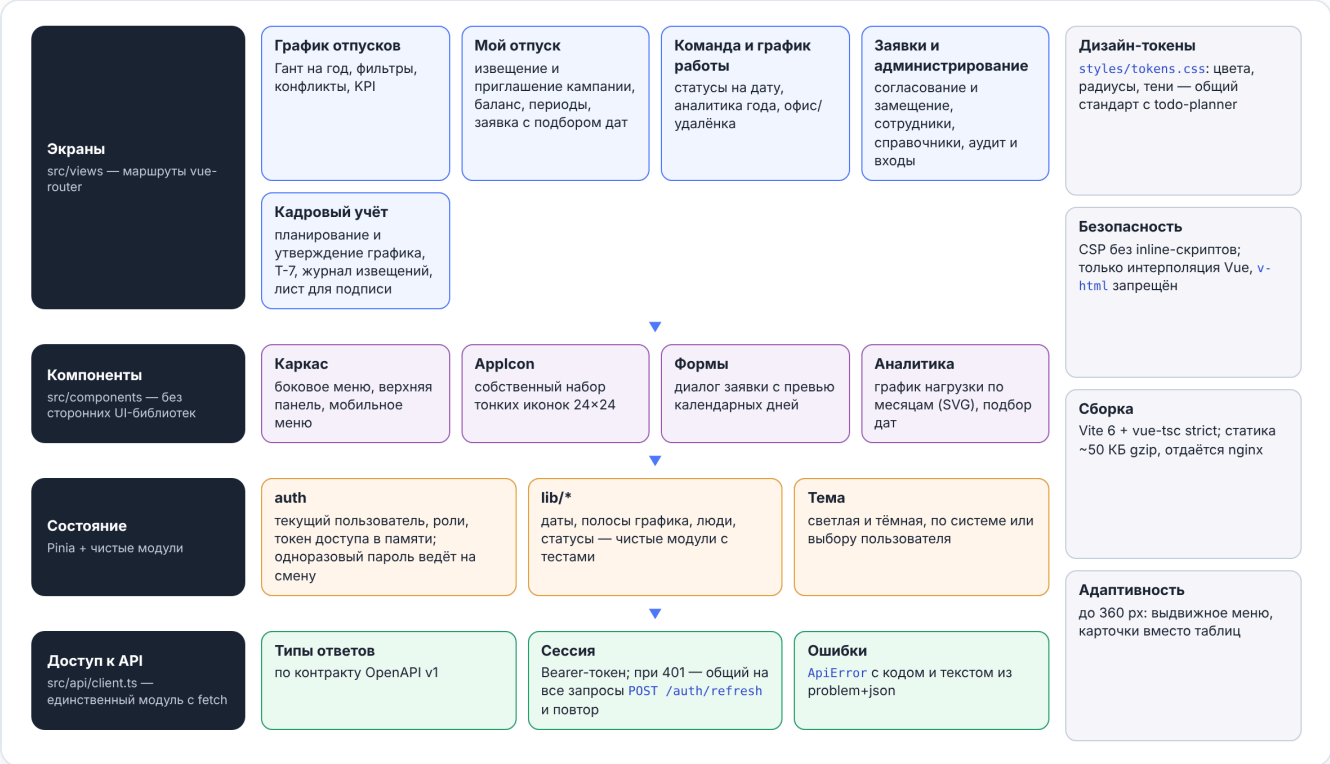


Рис. 7. Слои клиента

Клиент — Vue 3 + TypeScript strict + Vite + Pinia без сторонних UI-библиотек: дизайн-система (токены, иконки, паттерны) общая с внутренними продуктами todo-planner и almi_license. Единственный модуль, обращающийся к сети, — `web/src/api/client.ts`: он добавляет Bearer-токен, при `401` один раз обновляет его по httpOnly-cookie (параллельные запросы ждут общий результат обновления, иначе ротация отозвала бы токен) и превращает problem+json в `ApiError`.

ЭКРАН	НАЗНАЧЕНИЕ
График отпусков	Гант года: масштабы год/квартал/месяц, группы отделов, выходные и праздники, линия «сегодня», полоса нагрузки, слой удалёнки, сравнение выбранных сотрудников, выгрузка в Excel
Мой отпуск	Извещение о начале отпуска с кнопкой «Ознакомлен», приглашение кампании планирования с прогрессом, баланс с разбивкой, ближайший отпуск, год одним взглядом, заявка с подбором дат, перенос и отмена

ЭКРАН	НАЗНАЧЕНИЕ
Команда	Кто сегодня в офисе, на удалёнке, в отпуске; аналитика года: выводы, нагрузка по месяцам, конфликты, распределение дней
График работы	Недельный шаблон офис/удалёнка и разовые исключения на календаре
Заявки	Очередь «ждут моего решения» (своя команда, замещение, эскалированные) с пересечениями в отделе, отклонение с причиной, оформление отпуска за подчинённого
Кадровый учёт	Кампания планирования: срок, прогресс по отделам и сотрудникам, напоминание; утверждение графика и форма Т-7; журнал извещений с выгрузкой и листом для печати под подпись
Администрирование	Сотрудники (учётная запись: блокировка, сеансы, сброс пароля; замещение руководителя), справочники и производственный календарь, правила организации, журнал аудита и журнал входов
Профиль	Тема, письма, смена пароля, свои сеансы на устройствах, замещение на время отсутствия; экран смены одноразового пароля при первом входе

Тема оформления — светлая, тёмная или по настройке системы; интерфейс адаптирован до ширины 360 px.

7 Модель данных

Схема описана в `server/prisma/schema.prisma` (24 таблицы PostgreSQL); полный словарь данных и инварианты — документ 4.

7.1 Организация

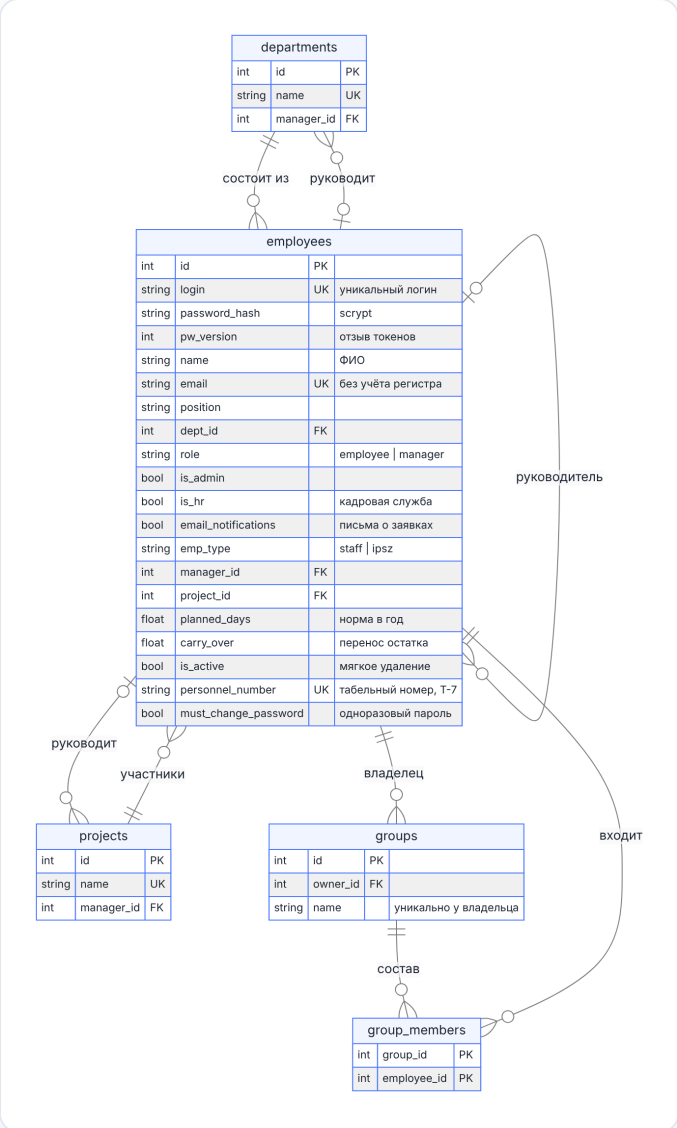


Рис. 8. ER: сотрудники, отделы, проекты, группы

7.2 Отпуска и режим работы

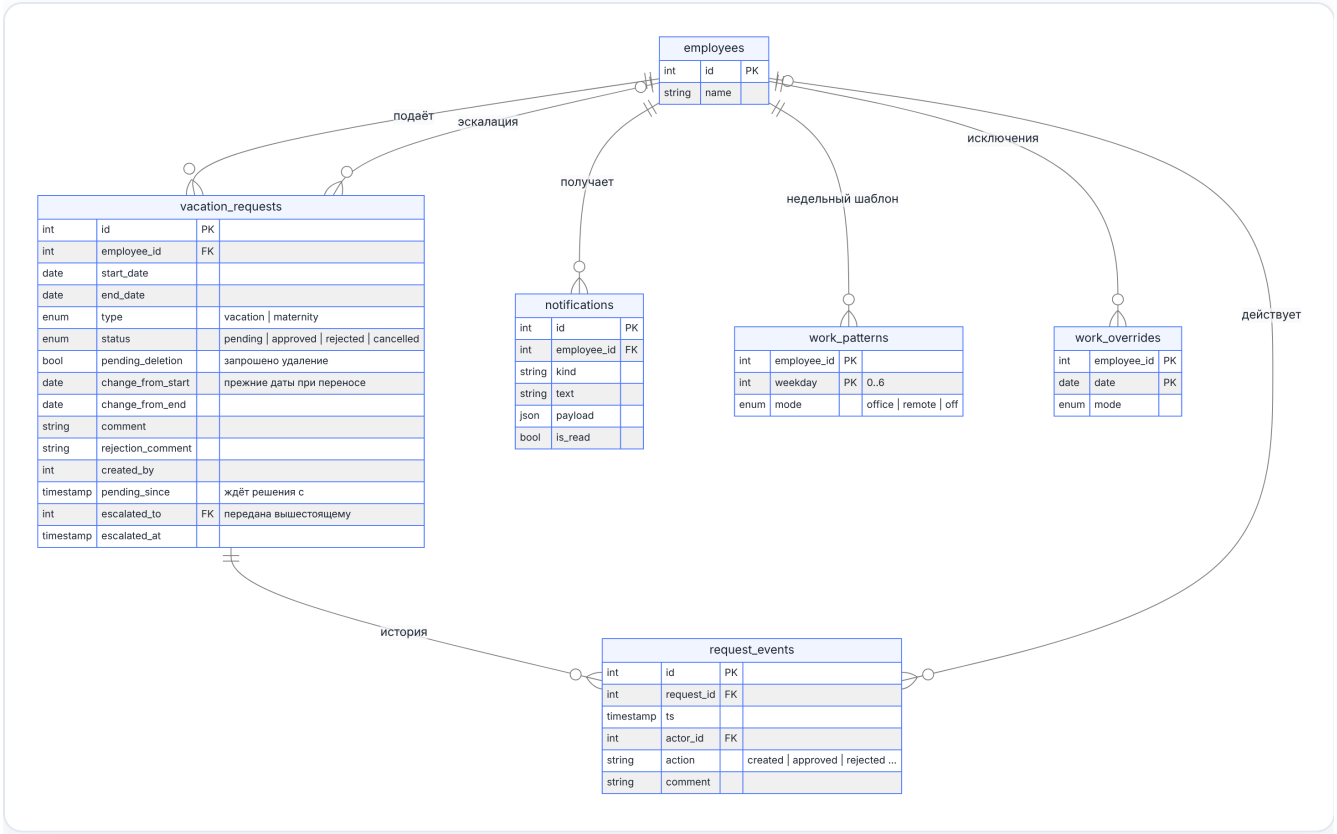


Рис. 9. ER: заявки, история, уведомления, режим работы

7.3 Служебные данные

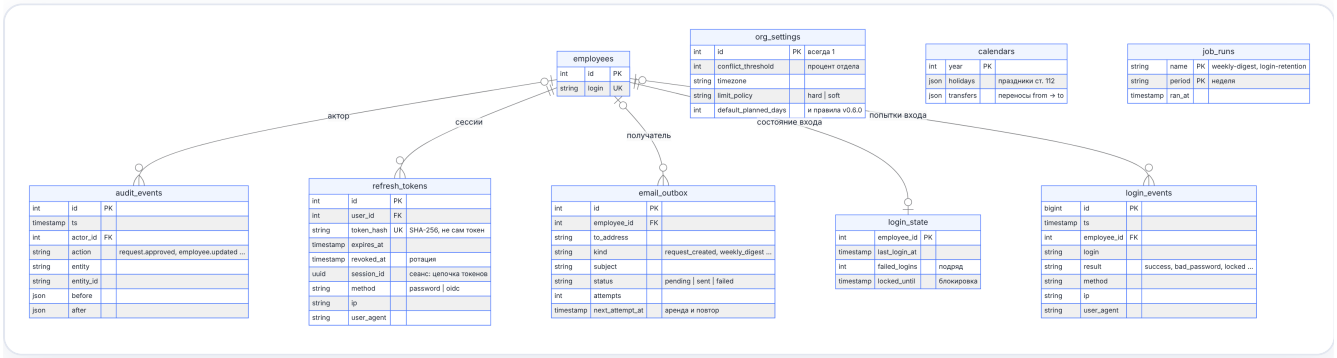


Рис. 10. ER: аудит, сессии, настройки, календарь

7.4 Кадровый учёт

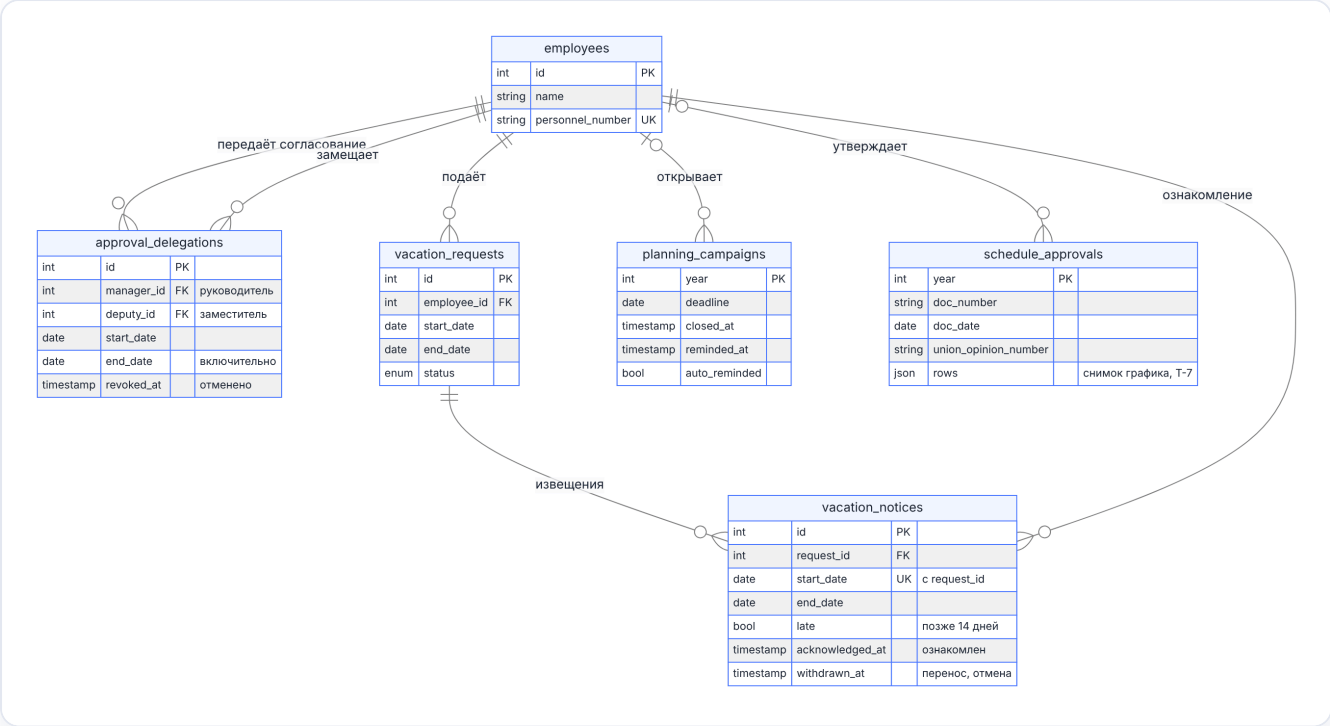


Рис. 11. ER: замещение, извещения, кампания и утверждённый график

7.5 Жизненный цикл заявки

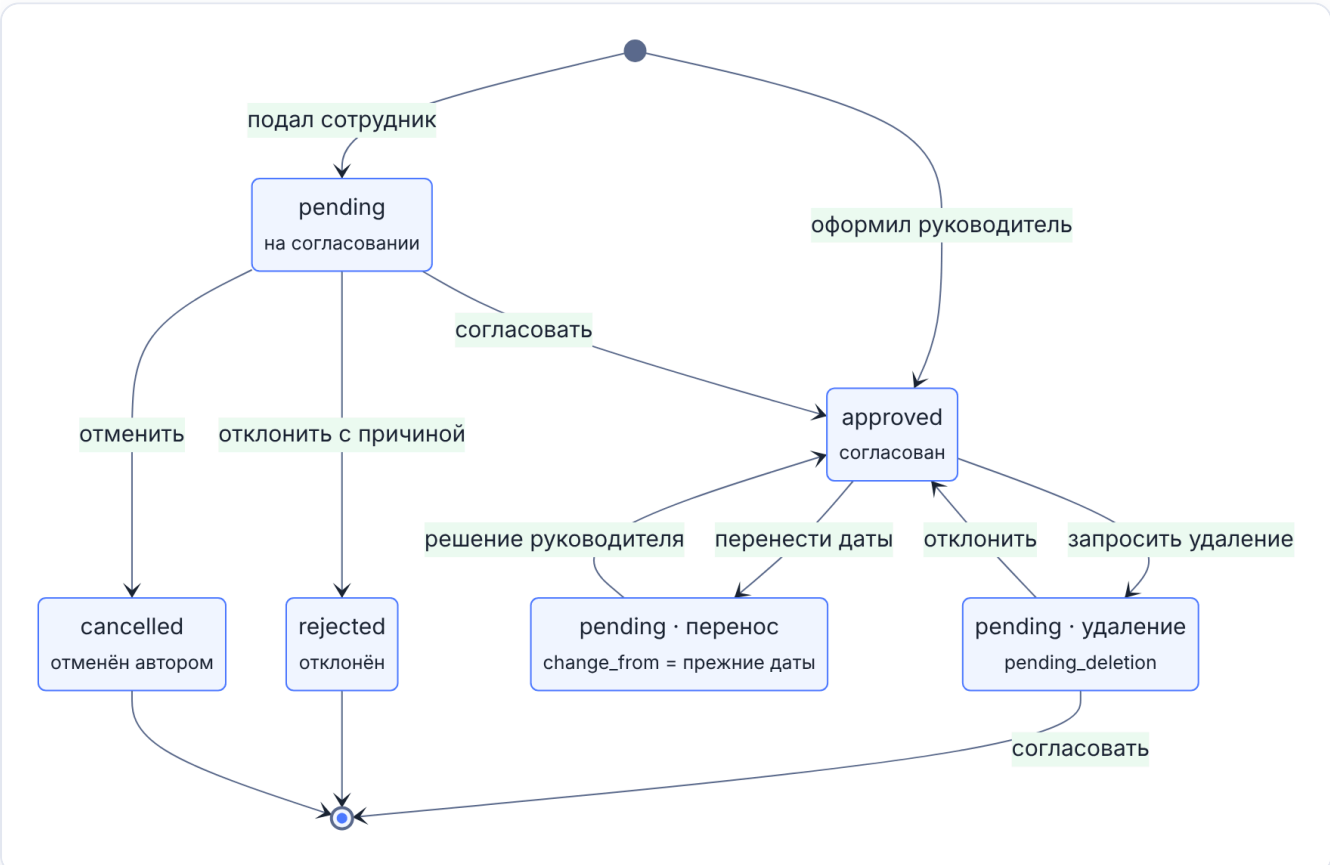


Рис. 12. Состояния заявки

8 Ключевые сценарии

8.1 Вход и продление сессии

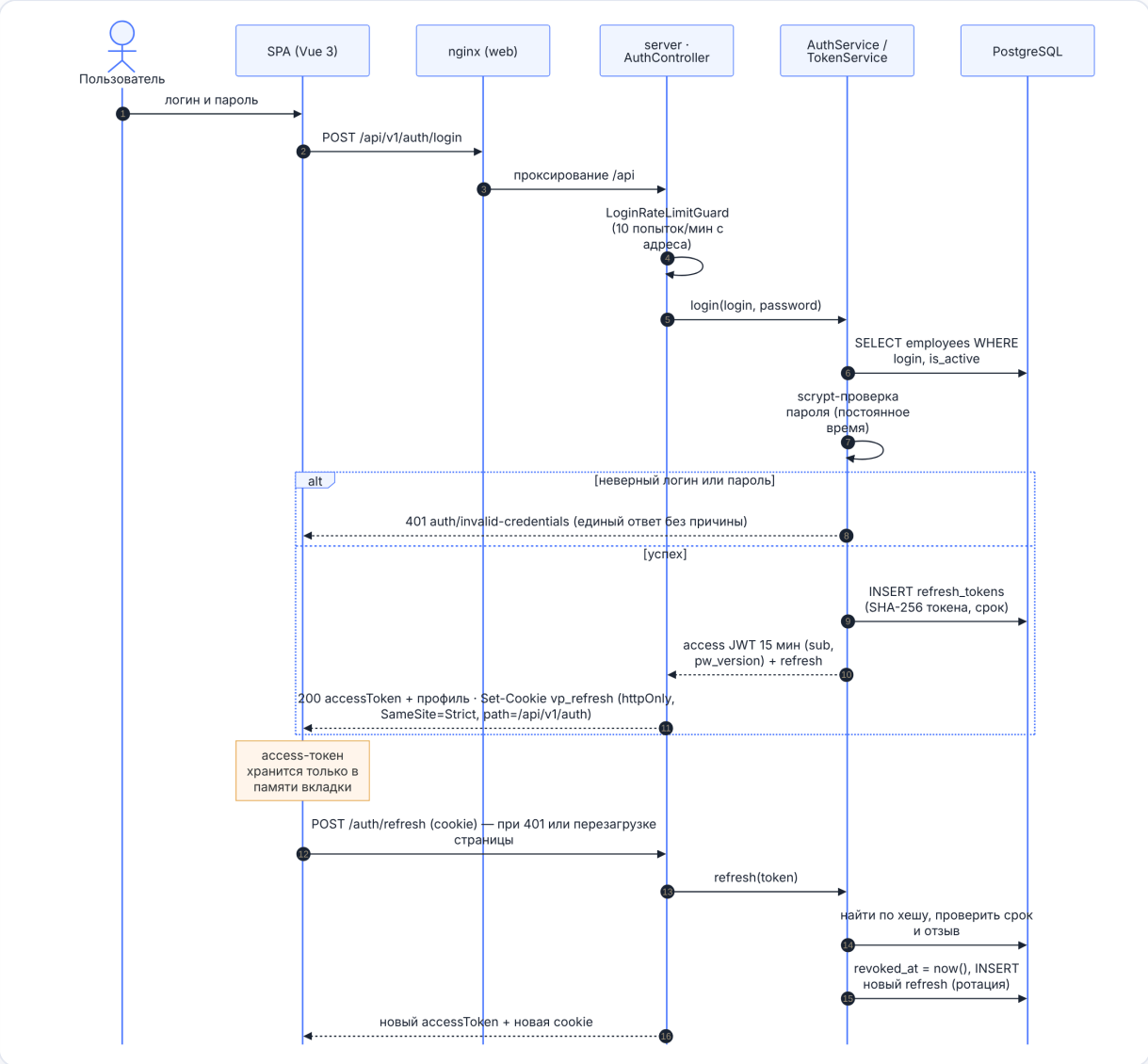


Рис. 13. Последовательность входа и обновления токена

8.2 Подача заявки

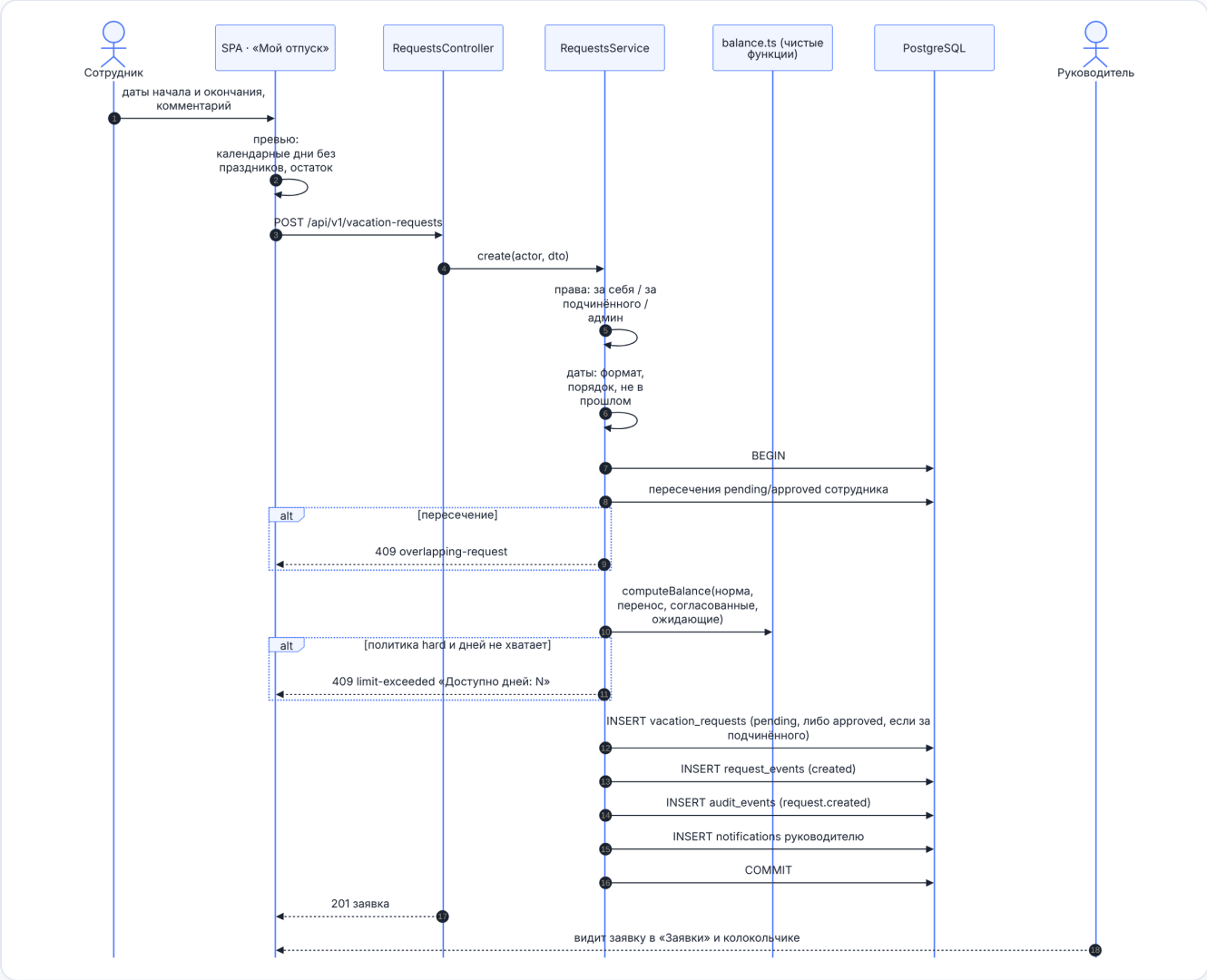


Рис. 14. Создание заявки: проверки и транзакция

8.3 Согласование и отклонение

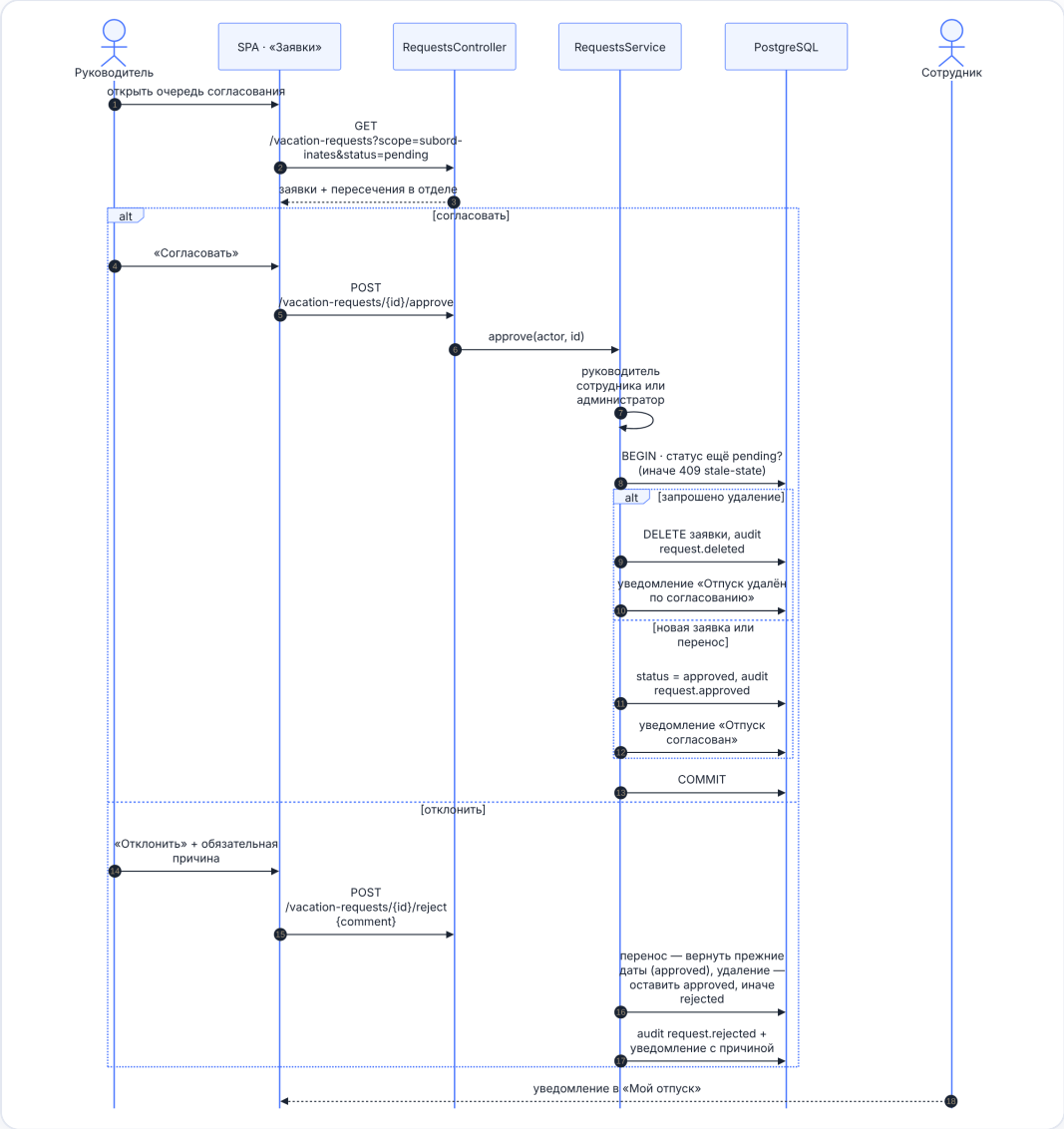


Рис. 15. Решение руководителя

8.4 Построение графика

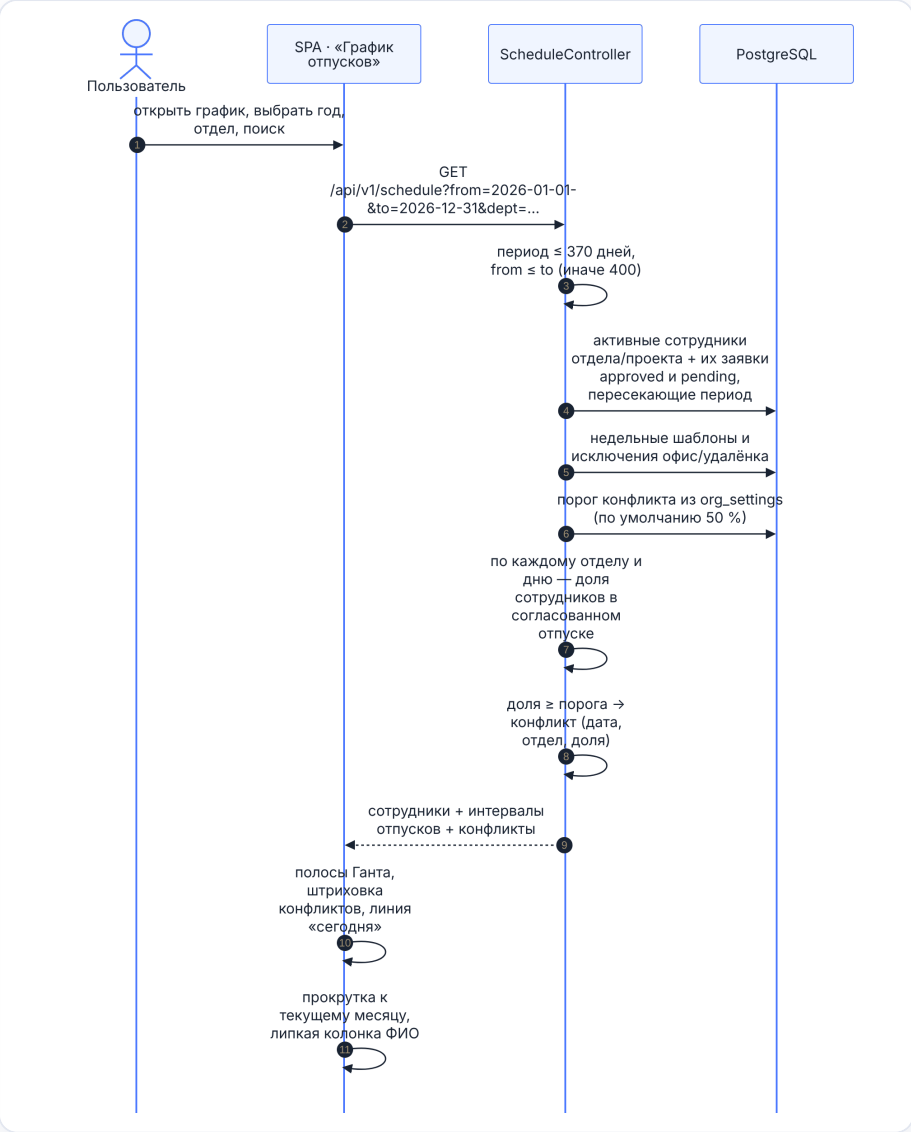


Рис. 16. Загрузка графика с конфликтами отделов

9 Безопасность

9.1 Аутентификация и сессии

МЕХАНИЗМ	РЕАЛИЗАЦИЯ
Хранение паролей	scrypt с солью в пуле потоков, сравнение за постоянное время (`auth/passwords.ts`); для несуществующего логина проверяется подставной хеш — время ответа не выдаёт, есть ли логин; пароли не попадают в журналы, ответы и выгрузки
Access-токен	JWT HS256, 15 минут, в памяти вкладки (не в localStorage)
Refresh-токен	Случайный, в httpOnly-cookie `vp_refresh` (SameSite=Strict, Secure в production, path `/api/v1/auth`); в БД хранится только SHA-256
Ротация	Каждое обновление атомарно отзывает использованный refresh и выдаёт новый; повтор того же токена в течение 30 секунд (параллельные вкладки) получает пару того же сеанса, позже — считается кражей: сеанс завершается целиком, в журнале входов — `refresh_reuse` (ADR-015)
Сеансы	Цепочка refresh-токенов одного входа (`session_id`), в access-токене — `sid` ; сотрудник видит свои сеансы (устройство, адрес, время) и завершает любой или все остальные, кадровая служба — все сеансы сотрудника; завершённый сеанс теряет доступ и поток событий не позже чем через 10 секунд
Блокировка	После `lockoutThreshold` неудачных попыток подряд (по умолчанию 10; 0 — выключено) учётная запись блокируется на `lockoutMinutes` (по умолчанию 15): `423 auth/account-locked` только при верном пароле, иначе обычный `401` ; сотрудник получает уведомление, кадровая служба снимает блокировку в карточке

МЕХАНИЗМ	РЕАЛИЗАЦИЯ
Политика паролей	Не короче `passwordMinLength` (8–64), без логина и фамилии, не из распространённых (с учётом русской раскладки и цифр в конце), не последовательность клавиш или цифр, не из одного-двух символов, не совпадает с текущим — `400 validation/weak-password` (`auth/password-policy.ts`)
Одноразовый пароль	Пароль, выданный при создании, импорте или сбросе, нужно сменить при первом входе; сброс выдаёт сервер, кадровая служба видит пароль один раз
Смена пароля и увольнение	Смена пароля увеличивает `pw_version` , увольнение снимает `is_active` : выданные токены перестают действовать (ответ `401 auth/token-expired` , не позже 10 секунд в других процессах)
Ограничение частоты	10 попыток в минуту на пару «адрес + логин» и 60 на адрес → `429` с `Retry-After` ; окна — в PostgreSQL, общие для всех процессов и экземпляров (скользящая оценка по текущей и прошлой минуте); адрес клиента берётся из `X-Forwarded-For` только от доверенных прокси частных сетей (`TRUST_PROXY`)
Поток событий	Открывается по одноразовому билету на 60 секунд (отдельная аудитория JWT): токен не попадает в адрес и журналы прокси; при открытии сверяются версия пароля и активность
Ответ на неверный вход	Единый `401 auth/invalid-credentials` без указания причины
Единый вход (OIDC)	Authorization code + PKCE (S256); state, nonce и verifier — в подписанной httpOnly-cookie на 10 минут; id_token проверяется по JWKS (подпись, издатель, получатель, срок, nonce); сотрудник — по логину из утверждения; после входа — та же refresh-cookie
Отключение пароля	`LOCAL_LOGIN=false` : пароль остаётся только у администраторов — аварийный вход при недоступности провайдера

9.2 Защита запросов

Глобальные guards применяются ко всем маршрутам по умолчанию; публичные (`/healthz` , `/readyz` , `/metrics` , календарь, вход, обновление, поток событий по билету) отмечены явно. `/metrics` не проксируется nginx наружу и может требовать `METRICS_TOKEN` . DTO-валидация отбрасывает неизвестные поля. Ответы API содержат `X-Content-Type-Options: nosniff` , `X-Frame-Options: DENY` , `Referrer-Policy` , `Permissions-Policy` . nginx отдаёт SPA с Content Security Policy (`script-src 'self'` , без inline-скриптов, `frame-ancestors 'none'`); e2e-тесты выполняются под той же политикой и падают при её нарушении. В клиенте запрещён `v-html` : данные пользователей выводятся только интерполяцией Vue с автоэкранированием.

9.3 Модель авторизации

РОЛЬ	ПРАВА
Сотрудник	Свои заявки, баланс и график работы; общий график; команда и аналитика своего отдела; подбор дат для себя; свои извещения и ознакомление с ними; свои сеансы
Руководитель	Дополнительно: согласование заявок и оформление отпусков прямых подчинённых (руководитель отдела — и сотрудников отдела без руководителя), их режим работы, аналитика отделов, где работают подчинённые или которыми руководит; передача согласования заместителю на период
Заместитель	На период замещения — права руководителя по заявкам его команды: очередь, согласование и отклонение, уведомления о новых; свою заявку не согласует
Вышестоящий по эскалации	Решение по заявке, переданной ему после `escalationDays` дней без решения
Кадровая служба	Сотрудники, импорт, оргструктура, производственный календарь, выгрузки и аналитика всей организации, журнал аудита и журнал входов; блокировки, сеансы и сброс паролей сотрудников; замещение за руководителя; кампания планирования, утверждение графика и форма Т-7; журнал извещений. Не меняет правила организации, не согласует заявки, не меняет учётные записи администраторов и коллег по службе

РОЛЬ	ПРАВА
Администратор	Всё: дополнительно правила организации, выдача служебных ролей, согласование любых заявок; последнего администратора разжаловать нельзя

Права служебных ролей читаются из БД на каждый запрос (`auth/access.ts`), а не из токена: снятие роли действует сразу.

9.4 Журналирование и аудит

Каждое изменение данных пишет запись в `audit_events` (кто, когда, действие, объект, значения до и после) в той же транзакции, что и само изменение: запись журнала не может потеряться или появиться без изменения. Журнал только дополняется. Администратор и кадровая служба просматривают его с фильтром по объектам и раскрытием изменённых полей.

Попытки входа пишутся в отдельный журнал `login_events`: успешный вход паролем и через единый вход, неверный пароль, неизвестный логин, блокировка, отказ по частоте, вход отключённым паролем, неизвестный пользователь единого входа, повтор refresh-токена — с адресом и браузером. Журнал не прерывает вход при ошибке записи; срок хранения — `loginRetentionDays` (по умолчанию 365 дней), старые записи удаляет ежедневная задача (ADR-015).

9.5 Чек-лист безопасности

Сводка для приёмки службой ИБ (E62-05): требование, как оно выполнено и чем подтверждается. Тесты — в `server/test` и `web/e2e`, выполняются в CI на каждый push.

№	ТРЕБОВАНИЕ	КАК ВЫПОЛНЕНО	ПОДТВЕРЖДЕНИЕ
1	Идентификация и аутентификация	Пароль или единый вход OIDC (authorization code + PKCE, проверка id_token по JWKS); единый ответ `401` без причины; `LOCAL_LOGIN=false` оставляет пароль только администраторам	`auth.e2e`, `oidc.e2e`
2	Хранение паролей	scrypt с солью в пуле потоков, сравнение за постоянное время, подставной хеш для неизвестного логина; пароли не попадают в журналы, ответы и выгрузки	`passwords.unit`, `auth.e2e`
3	Политика паролей	Длина — правило организации (8–64), запрет логина, фамилии, распространённых паролей, последовательностей и текущего пароля	`password-policy.unit`, `security.e2e`

№	ТРЕБОВАНИЕ	КАК ВЫПОЛНЕНО	ПОДТВЕРЖДЕНИЕ
4	Первичный пароль	Выданный кадровой службой пароль — одноразовый: до смены API отвечает `403 auth/password-change-required`; сброс генерирует сервер	`security.e2e`, e2e «одноразовый пароль»
5	Защита от подбора	Лимит частоты (10 в минуту на адрес и логин, 60 на адрес) и временная блокировка учётной записи после серии неудачных попыток с уведомлением сотрудника	`auth.e2e`, `security.e2e`, лимит на двух экземплярах
6	Управление сеансами	Access 15 минут в памяти вкладки; refresh в httpOnly-cookie (SameSite=Strict, Secure); атомарная ротация и обнаружение повтора; список и завершение сеансов сотрудником и кадровой службой; смена пароля и увольнение отзывают все сеансы	`auth.e2e`, `security.e2e`, e2e «сброс пароля»
7	Разграничение доступа	Роли и права объектов проверяются на сервере в каждом запросе, служебные права читаются из БД; заместитель и эскалация — по тем же правилам, что очередь согласования	`staff.e2e`, `requests.e2e`, `delegation.e2e`
8	Регистрация событий	Журнал входов с адресом и браузером, срок хранения; журнал изменений в транзакции изменения, только дополняется; выгрузки отчётов — в журнале изменений	`security.e2e`, `audit-fixes.e2e`
9	Защита от веб-атак	Параметризованный SQL (Prisma, `$queryRaw` с параметрами); DTO-валидация с отбрасыванием неизвестных полей; CSP без inline-скриптов, `X-Frame-Options`, `nosniff`; `v-html` запрещён; refresh-cookie SameSite=Strict и Bearer-токен против CSRF	e2e «CSP стенда не нарушается», `contract.e2e`
10	Обработка ошибок	Только problem+json с кодом из каталога; внутренние ошибки — `500` без деталей, детали — в журнале сервера	`contract.e2e`, каталог ошибок (документ 2)
11	Конфиденциальность	Токен потока событий — одноразовый билет вне адреса; адреса и браузеры — только в журнале входов, доступ — кадровой службе; выгрузки без логинов и паролей	`events.e2e`, `security.e2e`
12	Секреты	Только из окружения; `JWT_SECRET` не короче 32 символов проверяется при старте; `METRICS_TOKEN` для метрик	`deployment-guide.md`, старт сервера

№	ТРЕБОВАНИЕ	КАК ВЫПОЛНЕНО	ПОДТВЕРЖДЕНИЕ
13	Состав и лицензии компонентов	SBOM CycloneDX 1.5, лицензионный гейт (copyleft и неизвестные лицензии — блокер), новые зависимости — через `docs/decisions.md`	задание CI «SBOM и лицензии», документ 11
14	Резервное копирование и восстановление	Ежедневный `pg_dump`, реплика и архив WAL, переключение на реплику по инструкции (ADR-014)	задание CI «Реплика PostgreSQL, архив WAL и переключение»
15	Устойчивость к нагрузке	500 одновременных пользователей на 5000 в базе без ошибок; при перегрузке — `503` с `Retry-After`	`docs/progress/load-v0.6.0.md`
16	Обновление без потери данных	Миграции применяются при старте; CI проверяет обновление с данными предыдущей версии (`scripts/upgrade-check.sh`)	шаг CI «Обновление с прошлого выпуска на данных»

Настраивает организация при развёртывании (руководство по развёртыванию): TLS-сертификат и перенаправление на HTTPS на nginx или балансировщике, провайдер единого входа, SMTP, отдельное хранилище резервных копий и архива WAL, сетевой доступ к `/metrics` только для мониторинга, правила организации по паролям и блокировке.

10 Программный интерфейс

REST API v1 — JSON по HTTPS с префиксом `/api/v1`, описание — OpenAPI 3.1 в `contracts/`. Реализация публикует интерактивную спецификацию на `/api/docs`. Полный каталог операций с правами и кодами ошибок — документ 2.

ГРУППА	ОСНОВНЫЕ ОПЕРАЦИИ
Сессии	вход по паролю и через единый вход, обновление, выход, профиль и личные настройки, смена пароля, политика паролей, свои сеансы и их завершение
Заявки	список (свои, подчинённых, все), создание, изменение и перенос, отмена или запрос удаления, согласование, отклонение
График и режим	график за период с конфликтами; шаблон и исключения режима работы
Аналитика	доступные отделы, обзор года, состав на дату, подбор дат
Справочники и настройки	отделы, проекты, группы, календарь (чтение и правка), правила организации, проверочное письмо
Импорт	сотрудники и остатки из XLSX/CSV и JSON, шаблон
Кадровый учёт	кампания планирования, прогресс, напоминание, утверждение графика, форма Т-7; извещения, ознакомление и журнал извещений; замещение руководителя; журнал входов, блокировка и сеансы сотрудника
Сервис	уведомления и поток событий (SSE), журнал аудита, выгрузка XLSX, пробы

11 Наблюдаемость и эксплуатация

ЗАДАЧА	СРЕДСТВО
Живость и готовность	`GET /healthz` (процесс), `GET /readyz` (запрос к БД); healthcheck контейнера PostgreSQL
Метрики	`GET /metrics` в формате Prometheus: задержки и коды по шаблонам маршрутов, задержка цикла событий, память и процессор, потоки SSE; из БД — соединения по состояниям, очередь писем, фоновые задачи, реплики и архив WAL. Процессы кластера складываются
Оповещения и панель	`deploy/monitoring/alerts.yml` (15 правил: доступность, 5xx, p95, перегрузка БД, реплика, архив, почта) и `grafana-dashboard.json`
Журналы	stdout контейнеров (`docker compose logs`), уровни log/warn/error; журнал входов и журнал изменений — в интерфейсе («Журнал аудита»)
Плановые задачи	Извещения, эскалация, напоминания о планировании, очистка журнала входов — в каждом процессе раз в `JOBS_POLL_SECONDS` (ADR-017); ошибки — в журнале сервера
Перегрузка	Нет свободного соединения или БД недоступна — `503 service-unavailable` с `Retry-After` , счётчик `vp_db_errors_total`
Резервное копирование	`backup` : `pg_dump -Fc` раз в сутки, 14 дней. С `docker-compose.replica.yml` — реплика, архив WAL и ежедневные физические копии (ADR-014)
Обновление	`git pull` и `docker compose up -d --build` ; миграции применяются при старте `server`
Целевые показатели	С репликой и архивом WAL: RPO — секунды (реплика) или до 5 минут (архив), RTO 15 минут. Без них: RPO 24 ч, RTO 1 ч. p95 ≤ 300 мс на чтение и ≤ 500 мс на запись при 500 одновременных пользователях

12 Технологический стек

СЛОЙ	ТЕХНОЛОГИИ
Сервер	Node.js 20/22, NestJS 11, TypeScript 5 strict, class-validator, exceljs
Данные	PostgreSQL 17, Prisma 6 (схема, миграции, клиент)
Клиент	Vue 3.5, Vite 6, Pinia, vue-router 4, собственные компоненты и иконки
Тесты	Vitest + supertest (API на тестовой БД), Vitest (модули клиента), Playwright (e2e интерфейса)
Качество	ESLint 9, vue-tsc, тест сверки контракта, лицензионный гейт SBOM (CycloneDX 1.5)
Поставка	Docker multi-stage, docker compose, nginx; GitHub Actions и Gitea Actions
Документация	Markdown → pandoc → PDF (Chromium) и DOCX; схемы Mermaid и HTML → PNG

13 Ключевые решения

РЕШЕНИЕ	АЛЬТЕРНАТИВА	ОБОСНОВАНИЕ
Модульный монолит NestJS	Микросервисы	Масштаб до 5000 пользователей обслуживается одним процессом; DI, guards и OpenAPI из коробки; модули повторяют предметные области
PostgreSQL с первой версии	SQLite с последующим переходом	Конкурентная запись, транзакции, репликация; нет миграционного долга
Prisma	TypeORM, SQL вручную	Типобезопасные запросы, схема как документация, миграции с ревью SQL
JWT + refresh с ротацией в БД	Серверные сессии	Сервер без состояния для горизонтального масштабирования; отзыв через таблицу
scrypt из `node:crypto`	bcrypt, argon2	Без нативных зависимостей, достаточная стойкость
Свой UI без библиотек	PrimeVue, Quasar	Единая дизайн-система продуктов; минимальный SBOM для реестра
Аудит в транзакции изменения	Асинхронные события	Журнал всегда согласован с данными
Аналитика на сервере чистыми функциями	Расчёт в браузере	Одинаковые правила для всех клиентов; тестируемость; данные других отделов не уходят в браузер
Общее состояние в PostgreSQL (ADR-012)	Redis	Одно хранилище в поставке и резервном копировании; нагрузки 5000 пользователей хватает с запасом
SSE и опрос БД (ADR-013)	WebSocket, брокер сообщений	Сигналу «есть изменения» не нужен двусторонний канал; без брокера — меньше компонентов

РЕШЕНИЕ	АЛЬТЕРНАТИВА	ОБОСНОВАНИЕ
Реплика и архив WAL штатно (ADR-014)	Patroni, repmgr	RPO и RTO требований достигаются без сервиса согласования и лишних компонентов
Свой реестр метрик	prom-client	Нужна малая часть библиотеки; сложение процессов кластера всё равно требует своей обвязки
Сеансы — цепочки refresh-токенов, блокировка в `login_state` (ADR-015)	Серверные сессии, блокировка по адресу	Без нового хранилища; блокировка не сбрасывает кэш графика; кража токена видна при первом повторе
Утверждённый график — снимок (ADR-016)	Пометка на заявках	Запланированная дата формы Т-7 неизменна, переносы видны по текущим заявкам
Плановые задачи с идемпотентностью в БД (ADR-017)	Отдельный планировщик, выбор лидера	Работают при любом числе процессов и экземпляров без новых компонентов

Полные записи — `docs/adr/`.

14 План развития

ВЕРСИЯ	СОДЕРЖАНИЕ	СТАТУС
v0.1.0	Зрелый стек вместо прототипа: вход, заявки, график, справочники, аудит, Docker, SBOM, CI	выпущена
v0.2.x	Схемы документации, XLSX-выгрузки, ограничение частоты входа, security-заголовки, полосы Ганта	выпущена
v0.3.0	Конвейер документов и новые схемы; новый интерфейс и иконки; праздники в балансе; аналитика пересечений и подбор дат; график работы, панель команды, сравнение; резервное копирование; e2e интерфейса и сверка контракта в CI	выпущена
v0.4.0	Интеграции: импорт сотрудников и остатков, единый вход OIDC, письма о заявках и еженедельная сводка, роль кадровой службы, календарь в интерфейсе	выпущена
v0.5.0	Эксплуатация на 5000: кластер процессов, кэш графика, нагрузочная валидация, метрики Prometheus, реплика и архив WAL, живые обновления, мгновенный отзыв сеанса	выпущена
v0.6.0	Готовность к приёмке: кампания планирования, утверждение графика и форма Т-7, извещение за две недели; замещение и эскалация согласования; журнал входов, блокировка, сеансы, политика паролей, чек-лист безопасности	текущая

ВЕРСИЯ	СОДЕРЖАНИЕ	СТАТУС
v1.0.0	Приёмка HR и ИБ, комплект для реестра отечественного ПО	план